Project Executable Files

Date	01 July 2026
Team ID	
Project Name	Rising Waters – Flood Risk Prediction and Monitoring System
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA *(Uses CSV dataset instead of a database)*
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	No
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

SMARTBRIDGE

SKILL WALLET
A SmartBridge Product

Step 2: File / Folder Structure

Provide an overview of your projects file and folder structure below:

```
RisingWaters/
├── app.py                          # Flask application (all routes)
├── requirements.txt
├── README.md
├── data/
│   └── flood_risk_data.csv         # Generated training dataset (5,200 records, 5 risk classes)
├── model/
│   ├── train_models.py             # ML training pipeline
│   ├── best_model.pkl              # Best model (XGBoost, 93.42%)
│   ├── scaler.pkl                  # StandardScaler
│   ├── label_encoder.pkl           # LabelEncoder (5 risk classes)
│   ├── feature_importance.json     # Feature importance (Random Forest)
│   └── model_meta.json             # Metrics, model comparison, risk profiles
├── templates/
│   ├── base.html                   # Shared layout & navbar
│   ├── index.html                  # Home page (Scenario overview)
│   ├── predict.html                # Scenario 1 – Flood Risk Prediction
│   ├── suitability.html            # Scenario 2 – Location Safety Assessment
│   └── research.html               # Scenario 3 – Research & Analytics Dashboard
├── static/
│   ├── css/
│   ├── js/
│   └── images/
├── utils/
│   ├── data_loader.py
│   ├── preprocess.py
│   └── evaluate.py
└── vercel.json                     # Vercel deployment configuration
```

Step 3: Deployment / Access Details

Field	Details
Login Credentials (Demo)	Not Applicable (No login required)
Platform / Hosting Provider	Vercel (Frontend Hosting), Flask (Backend API), GitHub (Source Code Repository)
Repository Link	https://github.com/sasivardhan3011/Rising-Waters
Demo Video Link	https://drive.google.com/file/d/1TDAqRizSs22koJ5lYH-6gaV1HjTeihQl/view?usp=sharing

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites: Install Python 3.x, Node.js, npm, Visual Studio Code, Git, and the required project dependencies. Ensure that an internet connection is available for accessing the deployed application or downloading the necessary packages.

Step 1: Clone or download the Rising Waters project from the GitHub repository.

Step 2: Install the required frontend and backend dependencies using npm install and pip install -r requirements.txt.

Step 3: Start the Flask backend by running python app.py.

Step 4: Start the React application using npm run dev or access the deployed application through the Vercel URL.

Step 5: Open the application in a web browser, select a location/monitoring zone, and enter environmental parameters (rainfall, river level, flow rate, temperature, humidity, wind speed, elevation, slope, soil type, etc.).

Step 6: View the predicted flood risk level, confidence score, top risk probabilities, safety recommendations, parameter gap analysis, and explore analytics on the Research Dashboard.

Step 5: Known Issues / Limitations

S. No	Known Issue / Limitation	Workaround / Status
1	The application uses a trained dataset and does not use real-time IoT sensor or live weather data.	Planned to integrate live weather APIs and IoT-based river gauge sensors in future versions.
2	Flood risk predictions depend on the quality and accuracy of the input parameters provided by the user.	Users should enter accurate and up-to-date environmental values to obtain reliable risk predictions.
3	The current version focuses on flood risk prediction and location safety assessment only.	Future updates will include evacuation route suggestions, early warning alerts, rainfall forecasting, and impact assessment.